BÀI THỰC HÀNH - NGÔN NGỮ DART

Bài thực hành 1: Tính điểm trung bình và Xếp loại

Yêu cầu:

1. Tạo một biến **`Map<String, double>`** tên là **`diemMonHoc`** để lưu điểm các môn học.
   - Ví dụ: {'Toán': 9.0, 'Văn': 7.5, 'Anh': 8.0}.
2. Viết một hàm **`double tinhDiemTrungBinh(List<double> diem)`** nhận vào một danh sách điểm và trả về điểm trung bình.
3. Viết một hàm **`String xepLoai(double dtb)`** nhận vào điểm trung bình (DTB) và trả về xếp loại học lực:
   - DTB $\geq$ 9.0: "Xuất sắc"
   - DTB $\geq$ 8.0: "Giỏi"
   - DTB $\geq$ 6.5: "Khá"
   - DTB $\geq$ 5.0: "Trung bình"
   - Dưới 5.0: "Yếu"
4. Trong hàm main(), hãy:
   - Lấy danh sách các điểm từ **`diemMonHoc.values`**.
   - Gọi hàm **`tinhDiemTrungBinh`** để tính DTB.
   - Gọi hàm **`xepLoai`** để lấy học lực.
   - In kết quả ra màn hình (ví dụ: "Điểm trung bình: 8.17. Xếp loại: Giỏi").

Bài thực hành 2: Lập trình Hướng Đối Tượng (OOP) - Quản lý Thư viện

Yêu cầu:

1. Tạo một abstract class tên là TaiLieu có các thuộc tính:
   - String maSo
   - String tenNhaXuatBan
   - int soLuongPhatHanh
   - Một phương thức trừu tượng: void **`hienThiThongTin`**();
2. Tạo 3 lớp con kế thừa (extends) từ TaiLieu:
   - **`class Sach`**: Thêm thuộc tính String tenTacGia, int soTrang.
   - **`class TapChi`**: Thêm thuộc tính int soPhatHanh, int thangPhatHanh.
   - **`class Bao`**: Thêm thuộc tính DateTime ngayPhatHanh.
3. Các lớp con bắt buộc phải **`@override`** phương thức **`hienThiThongTin()`** để in ra toàn bộ thông tin của tài liệu đó (bao gồm cả thông tin của lớp cha và thông tin riêng của lớp con).
4. Trong hàm main(), hãy:
   - Tạo một **`List<TaiLieu>`** (List chứa các đối tượng từ lớp cha).
   - Thêm vào List 1 đối tượng Sach, 1 đối tượng TapChi, và 1 đối tượng Bao (với dữ liệu tự chọn).
   - Dùng vòng lặp for...in để duyệt qua List và gọi phương thức **`hienThiThongTin()`** của từng phần tử.

Bài thực hành 3: Lập trình Bất đồng bộ (Async) với API thực tế

Yêu cầu:

Phần 1: Future, async, await với Public API

1. Import các thư viện cần thiết:
   `import 'package:http/http.dart' as http;`
   `import 'dart:convert';`
2. Viết hàm `Future<String> fetchApiData()`:
   - Sử dụng `async`.
   - Bên trong hàm, sử dụng `http.get()` để gọi một API.
     Ví dụ API: Uri.parse('https://jsonplaceholder.typicode.com/posts/1')
   - `await` kết quả trả về (response).
   - Kiểm tra nếu response.statusCode == 200 (thành công):
     return response.body; (trả về chuỗi JSON).
   - Nếu không, `throw Exception`('Tải dữ liệu thất bại!');
3. Viết hàm `Map<String, dynamic> parseData(String responseBody):`
   - Sử dụng `jsonDecode` (từ dart:convert) để chuyển chuỗi JSON (đã fetch được) thành một đối tượng `Map`.
   - Trả về `Map` đã parse.
4. Trong hàm main(), hãy:
   - Bao bọc các lệnh gọi trong khối try-catch.
   - Gọi `fetchApiData()` và await kết quả (là responseBody).
   - Gọi `parseData()` với `responseBody` để lấy `dataMap`.
   - In ra tiêu đề (title) và nội dung (body) từ dataMap.
     - Ví dụ: print('Tiêu đề: ${dataMap['title']}');

Phần 2: Stream, async*, yield

1. Viết hàm Stream<int> countDown():
   - Sử dụng async* (async-sao).
   - Tạo 1 vòng lặp for đếm ngược từ 5 về 1.
   - Trong mỗi vòng lặp, await 1 giây (Future.delayed(Duration(seconds: 1))).
   - Sử dụng yield để phát ra số đếm hiện tại.
2. Trong hàm main:
   - In ra: "Chuẩn bị đếm ngược...".
   - Dùng vòng lặp await for để lắng nghe (listen) countDown() Stream và in ra từng số nó phát ra.
   - Sau khi vòng lặp kết thúc, in "Kết thúc!".

Gợi ý một số Public APIs:

- **JSONPlaceholder (API giả lập):** https://jsonplaceholder.typicode.com/posts/1 (Lấy bài viết)
- **Cat Facts (API dữ liệu ngẫu nhiên):** https://catfact.ninja/fact

■ The end